

CS3401 - Algorithms

Topic: rabin karp algorithm

1. SUMMARY

The Rabin-Karp algorithm is a string searching algorithm that uses hashing to find any substring in a text. It is a probabilistic algorithm, meaning it can return false positives, but never false negatives. The core principle of the Rabin-Karp algorithm is to calculate the hash value of the pattern and the initial window of the text, then slide the window over the text, updating the hash value at each step. The algorithm has a time complexity of $O(n+m)$, where n is the length of the text and m is the length of the pattern. The Rabin-Karp algorithm has several types and classifications, including the basic Rabin-Karp algorithm, the modified Rabin-Karp algorithm, and the weighted Rabin-Karp algorithm. The algorithm has a wide range of real-world applications, including text search, plagiarism detection, data compression, and intrusion detection systems. The Rabin-Karp algorithm is critical in the context of CS3401 - Algorithms, as it is a fundamental algorithm for string searching. The algorithm has a historical background dating back to 1987, when it was first proposed by Richard M. Karp and Michael O. Rabin. The evolution of the algorithm has led to several variations and improvements, including the use of rolling hash functions and the incorporation of additional heuristics.

2. SHORT NOTES

The Rabin-Karp algorithm is a technical definition of a string searching algorithm that uses hashing to find any substring in a text. The algorithm works by calculating the hash value of the pattern and the initial window of the text, then sliding the window over the text, updating the hash value at each step. The algorithm uses a rolling hash function to efficiently update the hash value at each step. The rolling hash function is a complex example of a hash function that uses a polynomial equation to calculate the hash value.

There are several types and classifications of the Rabin-Karp algorithm, including:

- Basic Rabin-Karp algorithm: This is the original algorithm proposed by Karp and Rabin.
- Modified Rabin-Karp algorithm: This algorithm uses a different hash function, such as the weighted hash function.

- Weighted Rabin-Karp algorithm: This algorithm uses a weighted hash function to give more importance to certain characters in the pattern.

The nuanced step-by-step working principle of the Rabin-Karp algorithm is as follows:

1. Calculate the hash value of the pattern using a hash function.
2. Calculate the hash value of the initial window of the text using the same hash function.
3. Compare the hash values of the pattern and the initial window. If they match, then compare the characters of the pattern and the window to confirm the match.
4. Slide the window over the text, updating the hash value at each step using a rolling hash function.
5. Repeat steps 3 and 4 until the end of the text is reached.

The relevant formulas and equations for the Rabin-Karp algorithm are:

- Hash value calculation: $h = (h * d + c) \bmod q$, where h is the hash value, d is the base of the hash function, c is the character being added, and q is the modulus.
- Rolling hash function: $h = (h - c * d^{(m-1)}) * d + c$, where h is the hash value, c is the character being removed, d is the base of the hash function, and m is the length of the pattern.

The highly descriptive diagrams explanation for the Rabin-Karp algorithm includes:

- Diagram 1: A diagram showing the initial window of the text and the pattern, with the hash values calculated for each.
- Diagram 2: A diagram showing the sliding window of the text, with the hash values updated at each step.

The in-depth SWOT analysis for the Rabin-Karp algorithm is as follows:

- Advantages: The Rabin-Karp algorithm has a time complexity of $O(n+m)$, making it efficient for large texts. The algorithm is also simple to implement and understand.
- Disadvantages: The Rabin-Karp algorithm can return false positives, which can be a problem in certain applications. The algorithm also requires a good hash function to work efficiently.
- Opportunities: The Rabin-Karp algorithm can be used in a wide range of applications, including text search, plagiarism detection, data compression, and intrusion detection systems.
- Threats: The Rabin-Karp algorithm can be affected by the choice of hash function, which can lead to poor performance if not chosen carefully.

The 5-7 distinct real-world applications of the Rabin-Karp algorithm are:

- Text search: The Rabin-Karp algorithm can be used to search for a pattern in a large text.
- Plagiarism detection: The Rabin-Karp algorithm can be used to detect plagiarism in documents by searching for similar patterns.
- Data compression: The Rabin-Karp algorithm can be used to compress data by searching for repeated patterns.
- Intrusion detection systems: The Rabin-Karp algorithm can be used to detect intrusions in a network by searching for patterns of malicious activity.
- Bioinformatics: The Rabin-Karp algorithm can be used to search for patterns in DNA sequences.

The expert-level comparison with at least 3 related concepts is as follows:

- The Rabin-Karp algorithm is similar to the Knuth-Morris-Pratt algorithm, which also uses a rolling hash function to search for patterns in a text. However, the Knuth-Morris-Pratt algorithm uses a different approach to update the hash value at each step.
- The Rabin-Karp algorithm is similar to the Boyer-Moore algorithm, which also uses a rolling hash function to search for patterns in a text. However, the Boyer-Moore algorithm uses a different approach to update the hash value at each step and also uses a bad character heuristic to skip over characters in the text.
- The Rabin-Karp algorithm is similar to the Bloom filter algorithm, which also uses a hash function to search for patterns in a text. However, the Bloom filter algorithm uses a different approach to calculate the hash value and also uses a bit array to store the hash values.

The detailed Time and space complexity analysis for the Rabin-Karp algorithm is as follows:

- Time complexity: The time complexity of the Rabin-Karp algorithm is $O(n+m)$, where n is the length of the text and m is the length of the pattern.
- Space complexity: The space complexity of the Rabin-Karp algorithm is $O(1)$, as the algorithm only requires a constant amount of space to store the hash values and the pattern.

The dictionary of minimum 10 important terminologies for the Rabin-Karp algorithm is as follows:

- Hash function: A function that calculates a hash value for a given input.
- Rolling hash function: A hash function that efficiently updates the hash value at each step.

- Pattern: The string being searched for in the text.
- Text: The string being searched.
- Window: The initial window of the text being searched.
- Hash value: The value calculated by the hash function.
- Modulus: The value used to calculate the hash value.
- Base: The base of the hash function.
- Character: A single character in the pattern or text.
- False positive: A match that is not actually a match.

The current trends and future scope of the Rabin-Karp algorithm are as follows:

- The Rabin-Karp algorithm is still widely used in many applications, including text search and plagiarism detection.
- The algorithm is being improved and modified to work with new technologies, such as cloud computing and big data.
- The algorithm is being used in new applications, such as bioinformatics and intrusion detection systems.

3. PRACTICAL / CODING EXAMPLES

```
```c
```

```
void rabin_karp(char *text, char *pattern) {
 int n = strlen(text);
 int m = strlen(pattern);
 int h = 0;
 int p = 0;
 int d = 256;
 int q = 101;

 for (int i = 0; i < m; i++) {
 h = (h * d + pattern[i]) % q;
 p = (p * d + text[i]) % q;
```

```

 }

 for (int i = 0; i <= n - m; i++) {
 if (h == p) {
 int j;
 for (j = 0; j < m; j++) {
 if (text[i + j] != pattern[j]) {
 break;
 }
 }
 if (j == m) {
 printf("Pattern found at index %d\n", i);
 }
 }
 if (i < n - m) {
 p = (d * (p - text[i] * pow(d, m - 1)) + text[i + m]) % q;
 if (p < 0) {
 p = p + q;
 }
 }
 }
}

```

```

```python

```

```

def rabin_karp(text, pattern):

```

```

    n = len(text)

```

```

    m = len(pattern)

```

```

    h = 0

```

```

    p = 0

```

```

    d = 256

```

```

    q = 101

```

```
for i in range(m):
    h = (h * d + ord(pattern[i])) % q
    p = (p * d + ord(text[i])) % q

for i in range(n - m + 1):
    if h == p:
        j = 0
        while j < m:
            if text[i + j] != pattern[j]:
                break
            j += 1
        if j == m:
            print("Pattern found at index", i)
    if i < n - m:
        p = (d * (p - ord(text[i]) * pow(d, m - 1)) + ord(text[i + m])) % q
        if p < 0:
            p = p + q
```

Algorithm

1. Calculate the hash value of the pattern and the initial window of the text.
2. Compare the hash values of the pattern and the initial window. If they match, then compare the characters of the pattern and the window to confirm the match.
3. Slide the window over the text, updating the hash value at each step using a rolling hash function.
4. Repeat steps 2 and 3 until the end of the text is reached.

Time and Space Complexity

- Time complexity: $O(n+m)$, where n is the length of the text and m is the length of the pattern.
- Space complexity: $O(1)$, as the algorithm only requires a constant amount of space to store the hash values and the pattern.

4. IMPORTANT QUESTIONS

****Part-A****

Q1. Define Rabin-Karp algorithm and list its key components.

Answer: The Rabin-Karp algorithm is a string searching algorithm that uses hashing to find any substring in a text. The key components of the Rabin-Karp algorithm are:

1. Pattern (the substring to be searched)
2. Text (the string in which the substring is to be searched)
3. Hash function (used to calculate the hash value of the pattern and the text)
4. Modulus (a prime number used to avoid overflow)

Q2. Compare Rabin-Karp algorithm with Knuth-Morris-Pratt algorithm.

Answer: The Rabin-Karp algorithm and the Knuth-Morris-Pratt algorithm are both string searching algorithms. However, the main difference between them is that the Rabin-Karp algorithm uses hashing, while the Knuth-Morris-Pratt algorithm uses a lookup table to skip characters in the text. The Rabin-Karp algorithm has an average time complexity of $O(n+m)$, while the Knuth-Morris-Pratt algorithm has a time complexity of $O(n+m)$ in the worst case.

Q3. What is the purpose of using a modulus in the Rabin-Karp algorithm?

Answer: The purpose of using a modulus in the Rabin-Karp algorithm is to avoid overflow when calculating the hash value of the pattern and the text. The modulus is a prime number that is used to divide the hash value, thereby preventing it from exceeding the maximum limit of the data type.

Q4. Write a short note on the advantages of the Rabin-Karp algorithm.

Answer: The Rabin-Karp algorithm has several advantages, including:

1. Average time complexity of $O(n+m)$
2. Simple to implement
3. Can be used for multiple pattern searching
4. Can be used for searching in a text with multiple occurrences of the pattern

Q5. Give an example of the application of the Rabin-Karp algorithm in text editing software.

Answer: The Rabin-Karp algorithm can be used in text editing software to implement the "find" feature. For example, when a user searches for a word in a document, the Rabin-Karp algorithm can be used to quickly find all occurrences of the word in the document.

****Part-B****

Q1. Explain Rabin-Karp algorithm in detail with neat diagram and suitable examples.

Answer:

The Rabin-Karp algorithm is a string searching algorithm that uses hashing to find any substring in a text. The algorithm works as follows:

1. Calculate the hash value of the pattern and the first window of the text.
2. Compare the hash values of the pattern and the text.
3. If the hash values match, compare the characters of the pattern and the text to confirm the match.
4. If the hash values do not match, slide the window to the right and repeat steps 1-3.

The diagram shows the Rabin-Karp algorithm in action:

...

+-----+

| Pattern |

+-----+

| Hash Value |

+-----+

|

|

v

+-----+

| Text |

+-----+

| Hash Value |

+-----+

|

```
|
v
+-----+
| Compare |
+-----+
|
|
v
+-----+
| Match  |
+-----+
...
```

The Rabin-Karp algorithm has several advantages, including:

1. Average time complexity of $O(n+m)$
2. Simple to implement
3. Can be used for multiple pattern searching
4. Can be used for searching in a text with multiple occurrences of the pattern

The algorithm has several real-world applications, including:

1. Text editing software
2. Web search engines
3. Data compression

The disadvantages of the Rabin-Karp algorithm include:

1. High probability of false positives
2. Requires a good hash function to avoid collisions

Q2. Describe the types of Rabin-Karp algorithm with working principle and examples.

Answer:

The Rabin-Karp algorithm has several types, including:

1. Single-pattern Rabin-Karp algorithm: This type of algorithm searches for a single pattern in a text.
2. Multi-pattern Rabin-Karp algorithm: This type of algorithm searches for multiple patterns in a text.

3. Rabin-Karp algorithm with modulo arithmetic: This type of algorithm uses modulo arithmetic to avoid overflow when calculating the hash value.

The working principle of the Rabin-Karp algorithm is as follows:

1. Calculate the hash value of the pattern and the first window of the text.
2. Compare the hash values of the pattern and the text.
3. If the hash values match, compare the characters of the pattern and the text to confirm the match.
4. If the hash values do not match, slide the window to the right and repeat steps 1-3.

The following table compares the different types of Rabin-Karp algorithm:

Type	Description	Time Complexity
Single-pattern	Searches for a single pattern	$O(n+m)$
Multi-pattern	Searches for multiple patterns	$O(n+m)$
Modulo arithmetic	Uses modulo arithmetic to avoid overflow	$O(n+m)$

The Rabin-Karp algorithm has several real-world applications, including:

1. Text editing software
2. Web search engines
3. Data compression

Q3. Compare Rabin-Karp algorithm with related concepts. Discuss advantages and applications.

Answer:

The Rabin-Karp algorithm is related to other string searching algorithms, including:

1. Knuth-Morris-Pratt algorithm
2. Boyer-Moore algorithm
3. Brute force algorithm

The following table compares the Rabin-Karp algorithm with other string searching algorithms:

Algorithm	Time Complexity	Advantages	Disadvantages
---	---	---	---

Rabin-Karp	$O(n+m)$	Average time complexity, simple to implement	High probability of false positives
Knuth-Morris-Pratt	$O(n+m)$	Efficient, simple to implement	Limited to single-pattern searching
Boyer-Moore	$O(n+m)$	Efficient, simple to implement	Limited to single-pattern searching
Brute force	$O(nm)$	Simple to implement	Inefficient, high time complexity

The advantages of the Rabin-Karp algorithm include:

1. Average time complexity of $O(n+m)$
2. Simple to implement
3. Can be used for multiple pattern searching
4. Can be used for searching in a text with multiple occurrences of the pattern

The disadvantages of the Rabin-Karp algorithm include:

1. High probability of false positives
2. Requires a good hash function to avoid collisions

The Rabin-Karp algorithm has several real-world applications, including:

1. Text editing software
2. Web search engines
3. Data compression

****Part-C****

Q1. Elaborate on Rabin-Karp algorithm covering theory, implementation, real-world applications, and future scope.

Answer:

****Section 1: Deep theoretical background****

The Rabin-Karp algorithm is based on the concept of hashing, which is a technique used to map a large input to a smaller output. The algorithm uses a hash function to calculate the hash value of the pattern and the text. The hash function is designed to minimize collisions, which occur when two different inputs have the same output.

****Section 2: How it is implemented/used in practice****

The Rabin-Karp algorithm is implemented using a simple and efficient approach. The algorithm works as follows:

1. Calculate the hash value of the pattern and the first window of the text.
2. Compare the hash values of the pattern and the text.
3. If the hash values match, compare the characters of the pattern and the text to confirm the match.
4. If the hash values do not match, slide the window to the right and repeat steps 1-3.

****Section 3: Minimum 3 detailed real-world case studies or applications with explanation****

The Rabin-Karp algorithm has several real-world applications, including:

1. Text editing software: The Rabin-Karp algorithm can be used to implement the "find" feature in text editing software.
2. Web search engines: The Rabin-Karp algorithm can be used to search for keywords in web pages.
3. Data compression: The Rabin-Karp algorithm can be used to compress data by searching for repeated patterns.

****Section 4: Limitations and challenges****

The Rabin-Karp algorithm has several limitations and challenges, including:

1. High probability of false positives
2. Requires a good hash function to avoid collisions
3. Limited to single-pattern searching

****Section 5: Future scope and improvements****

The Rabin-Karp algorithm has several future scope and improvements, including:

1. Developing more efficient hash functions to minimize collisions
2. Improving the algorithm to search for multiple patterns
3. Developing more efficient algorithms for data compression

Q2. Critically analyze Rabin-Karp algorithm. Compare approaches, evaluate trade-offs, and justify with examples.

Answer:

****Section 1: Overview of the concept and its importance****

The Rabin-Karp algorithm is a string searching algorithm that uses hashing to find any substring in a text. The algorithm is important because it has several real-world applications, including text editing software, web search engines, and data compression.

****Section 2: Different approaches/techniques used****

The Rabin-Karp algorithm uses a simple and efficient approach to search for a pattern in a text. The algorithm works as follows:

- 1. Calculate the hash value of the pattern and the first window of the text.
- 2. Compare the hash values of the pattern and the text.
- 3. If the hash values match, compare the characters of the pattern and the text to confirm the match.
- 4. If the hash values do not match, slide the window to the right and repeat steps 1-3.

****Section 3: Detailed comparison of approaches with a table****

The following table compares the different approaches used in the Rabin-Karp algorithm:

Approach	Time Complexity	Advantages	Disadvantages
Simple hashing	$O(n+m)$	Average time complexity, simple to implement	High probability of false positives
Modulo arithmetic	$O(n+m)$	Efficient, simple to implement	Limited to single-pattern searching
Brute force	$O(nm)$	Simple to implement	Inefficient, high time complexity

****Section 4: Trade-off analysis (time, space, cost, complexity)****

The Rabin-Karp algorithm has several trade-offs, including:

- 1. Time complexity: The algorithm has an average time complexity of $O(n+m)$, which is efficient for large texts.
- 2. Space complexity: The algorithm requires a small amount of space to store the hash values and the pattern.
- 3. Cost: The algorithm is simple to implement and requires minimal resources.
- 4. Complexity: The algorithm is simple to understand and implement.

****Section 5: Justify the best approach with real examples****

The Rabin-Karp algorithm is the best approach for searching for a pattern in a text because it has an average time complexity of $O(n+m)$, which is efficient for large texts. The algorithm is also simple to implement and requires minimal resources.

****Section 6: Critical limitations and open problems****

The Rabin-Karp algorithm has several critical limitations and open problems, including:

1. High probability of false positives
2. Requires a good hash function to avoid collisions
3. Limited to single-pattern searching

The algorithm requires further research and development to improve its efficiency and effectiveness.

5. MCQs

Q1. What is the time complexity of the Rabin-Karp algorithm?

- a) $O(n^2)$
- b) $O(n+m)$
- c) $O(m^2)$
- d) $O(n*m)$

Answer: b) $O(n+m)$

Explanation: The time complexity of the Rabin-Karp algorithm is $O(n+m)$, where n is the length of the text and m is the length of the pattern. This is because the algorithm only requires a single pass over the text and pattern.

Q2. What is the space complexity of the Rabin-Karp algorithm?

- a) $O(n)$
- b) $O(m)$
- c) $O(1)$
- d) $O(n*m)$

Answer: c) $O(1)$

Explanation: The space complexity of the Rabin-Karp algorithm is $O(1)$, as the algorithm only requires a

constant amount of space to store the hash values and the pattern.

Q3. What is the purpose of the rolling hash function in the Rabin-Karp algorithm?

- a) To calculate the hash value of the pattern
- b) To calculate the hash value of the text
- c) To update the hash value at each step
- d) To compare the hash values of the pattern and the text

Answer: c) To update the hash value at each step

Explanation: The rolling hash function is used to update the hash value at each step, allowing the algorithm to efficiently slide the window over the text.

Q4. What is the advantage of the Rabin-Karp algorithm?

- a) It has a high time complexity
- b) It has a high space complexity
- c) It can return false positives
- d) It has a low time complexity

Answer: d) It has a low time complexity

Explanation: The Rabin-Karp algorithm has a low time complexity of $O(n+m)$, making it efficient for large texts.

Q5. What is the application of the Rabin-Karp algorithm?

- a) Text search
- b) Plagiarism detection
- c) Data compression
- d) All of the above

Answer: d) All of the above

Explanation: The Rabin-Karp algorithm has a wide range of applications, including text search, plagiarism detection, data compression, and intrusion detection systems.

Study Material - Generated by AI

7. YOUTUBE LINKS

- [Rabin Karp Algorithm Tutorial](https://www.youtube.com/results?search_query=rabin+karp+algorithm+tutorial)
- [String Searching using Rabin Karp Algorithm](https://www.youtube.com/results?search_query=string+searching+using+rabin+karp+algorithm)
- [Rabin Karp Algorithm for Pattern Searching](https://www.youtube.com/results?search_query=rabin+karp+algorithm+for+pattern+searching)
- [Implementation of Rabin Karp Algorithm in C](https://www.youtube.com/results?search_query=implementation+of+rabin+karp+algorithm+in+c)
- [Rabin Karp Algorithm for Text Search](https://www.youtube.com/results?search_query=rabin+karp+algorithm+for+text+search)